

SSL

Semantic Segmentation

Object Detection

June 2026

Usual Recipe

- Understand your task: domain, classes etc.
- Collect a lot of images accordingly.
- (Somehow) annotate those images.
- Use images and the labels (annotations) to train a (CNN) model.
- Deploy: use trained model from now on the test data.

Self-Supervised Learning

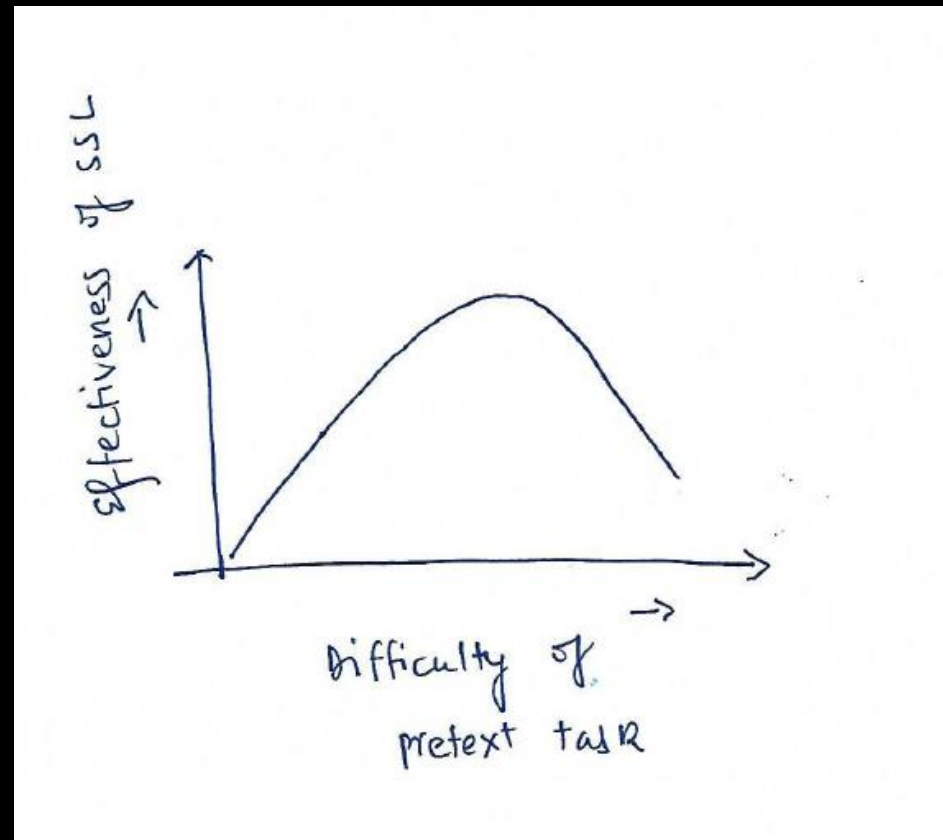
- Supervised learning tasks have pre-defined (and generally human-provided) labels.
- Unsupervised learning has just the data samples without any supervision, label or correct output.
- Self-supervised learning derives its labels from a co-occurring modality for the given data sample or from a co-occurring part of the data sample itself.

Pre-Text Task

- The pretext task is the self-supervised learning task solved to learn visual representations.
- E.g., Rotation of images
- Just think of your face image rotated at different angles.

Think how a plot “effectiveness of SSL” vs “difficulty of pretext task” will look?

Think how a plot “effectiveness of SSL” vs “difficulty of pretext task” will look?



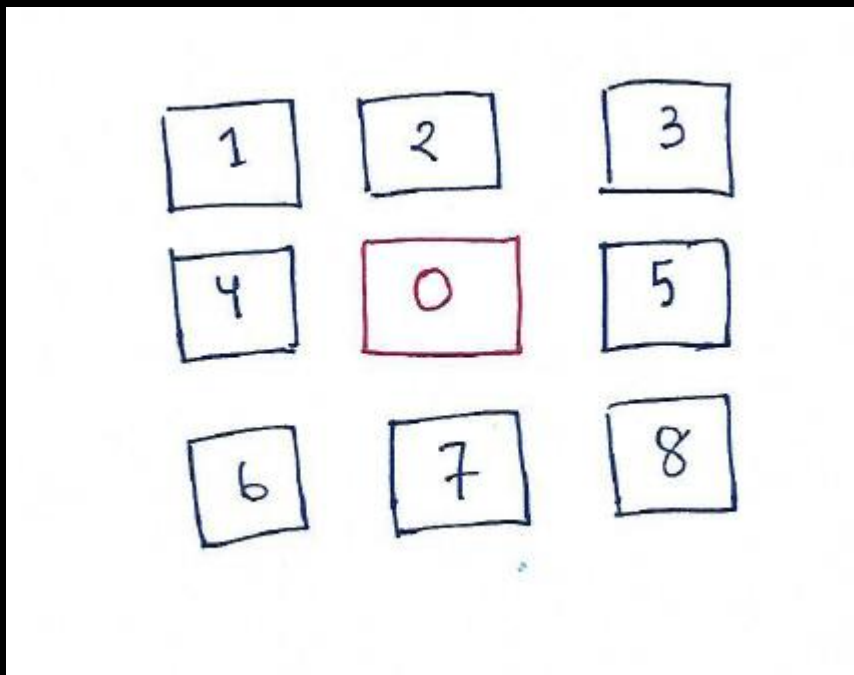
Context Prediction

- Key Idea: By predicting the relative position of image patches, the model learns to understand object structures and scene layouts.

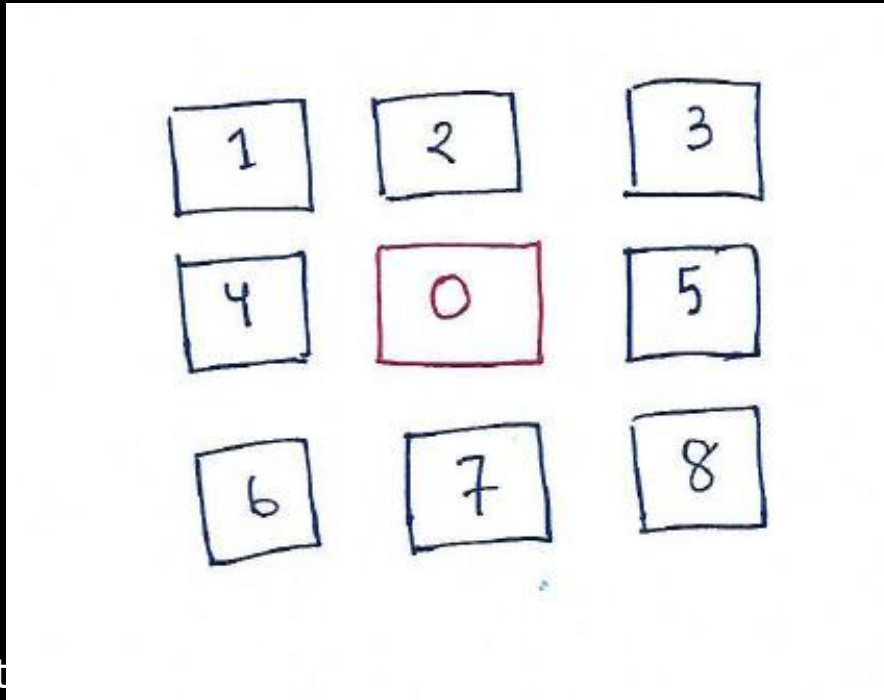
Context Prediction

- Patch Extraction: Extract a central patch and one of its eight neighboring patches. This results in pairs of patches with known spatial relationships.
- Prediction Task: Train a CNN to predict the position of the neighboring patch relative to the central patch. The network learns to classify the relative position into one of eight possible categories.

Context Prediction



Context Prediction



The algorithm

Context Prediction

- Does such context depend on the spatial resolution of images?
- In Earth observation, we work on a huge range of spatial resolution.

Image Segmentation

- *Split image into different regions*

Different regions usually cover the whole image

- Different regions usually do not overlap

- *Similarity predicate*

Satisfied by each region

- Not satisfied by union of different regions

- *Can be subjective*

Depending on how we define the notion of similarity

Before Deep Learning

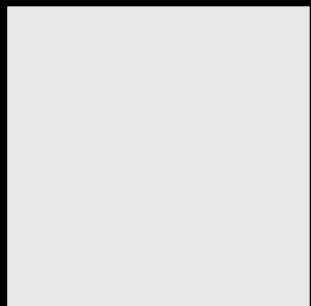
- *Region growing*
 - Start with one pixel of a potential region and try to grow until pixels being compared are too dissimilar
- *Clustering*
- *Split and merge*

Input Space for Clustering

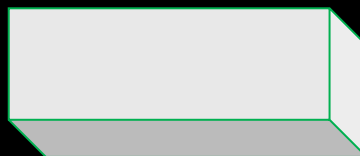
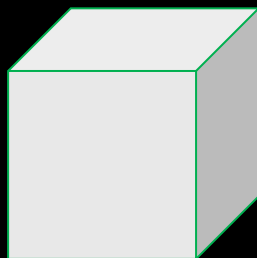
- Color
- Texture features

Typical Classification Network

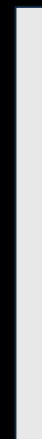
Input



Convolution layers

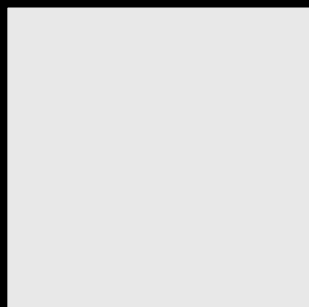


Fully connected layers

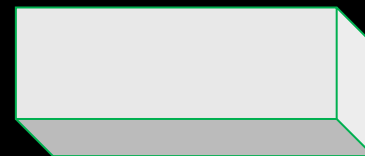
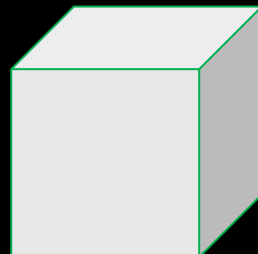


Fully Convolutional

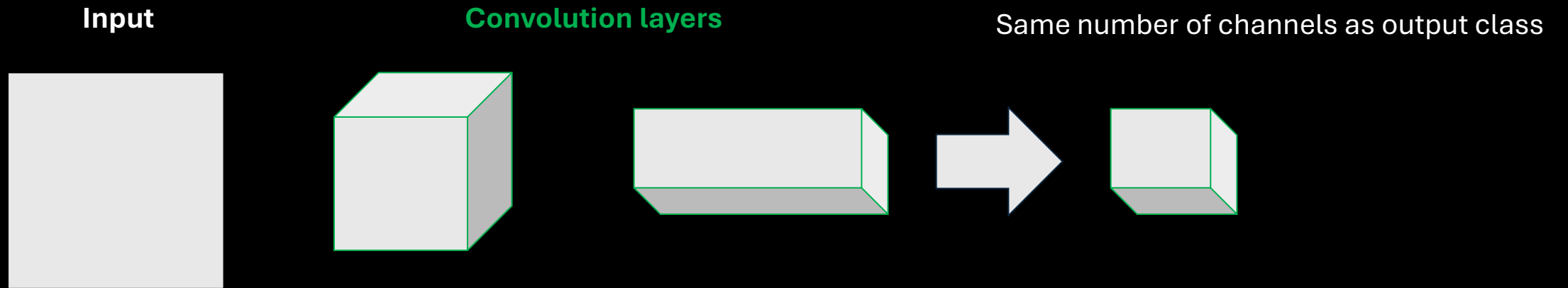
Input



Convolution layers

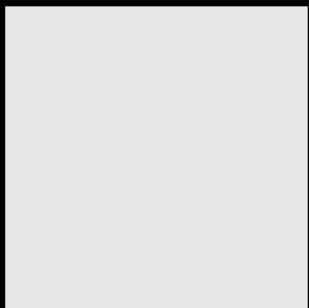


Fully Convolutional

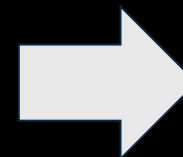
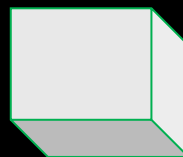
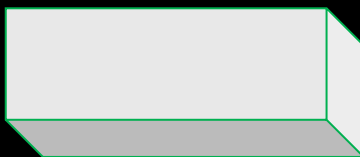
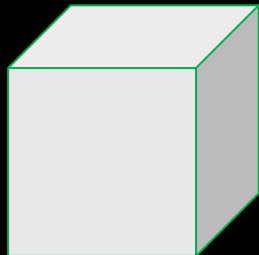


Fully Convolutional

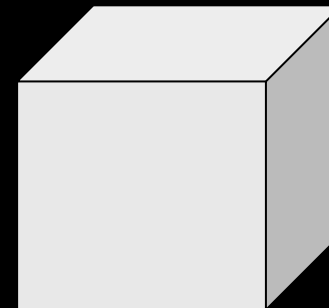
Input



Convolution layers



Output



Classifier to Semantic Segmentation

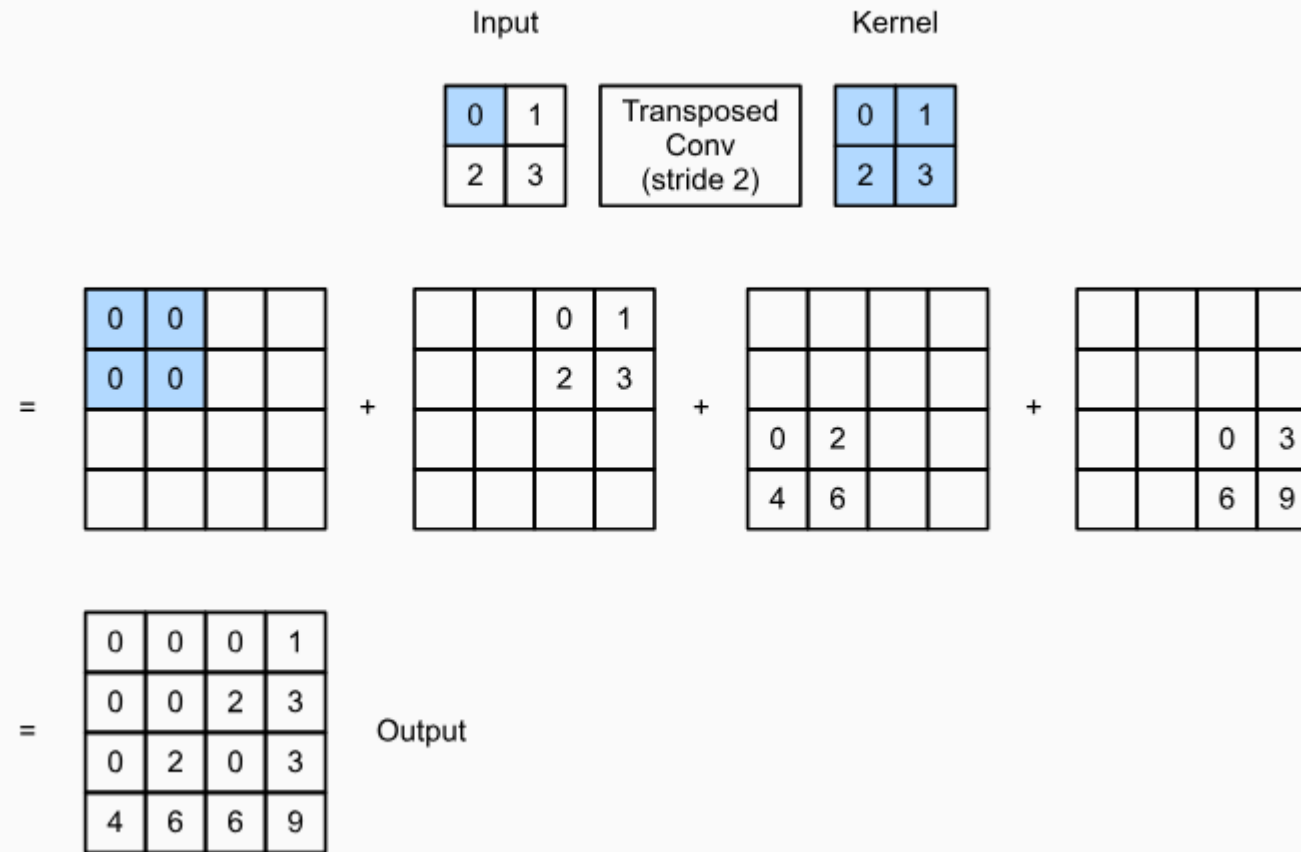
Convolutionalize the classification architectures: AlexNet, VGGNet

Remove classification layer

Use 1x1 convolution with required number of channel dimensions and upsample

Or replace the last step with transposed convolution.

Transposed Convolution



Transposed Convolution

```
# Input
input = torch.tensor([[0.0, 1.0], [2.0, 3.0]])
#Kernel
kernel1 = torch.tensor([[0.0, 1.0], [2.0, 3.0]])
kernel2 = torch.tensor([[4.0, 1.0], [2.0, 3.0]])

# Redefine the shape in 4 dimension
input = input.reshape(1, 1, 2, 2)
kernel1 = kernel1.reshape(1, 1, 2, 2)
kernel2 = kernel2.reshape(1, 1, 2, 2)
```

Transposed Convolution

```
# Transpose convolution Layer
transpose = nn.ConvTranspose2d(in_channels =1,
                              out_channels =1,
                              kernel_size=2,
                              stride = 2,
                              padding=0,
                              bias=False)

# Initialize Kernel
transpose.weight.data = kernel1
# Output value
output = transpose(input)

print(output)

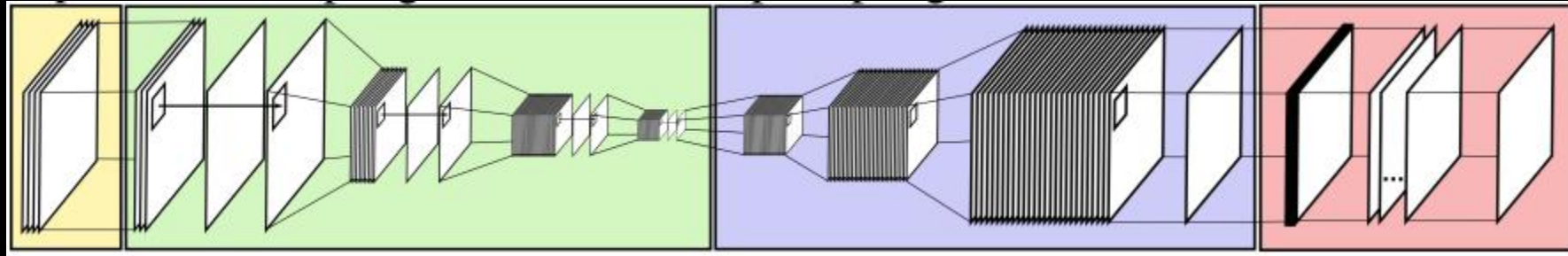
# Initialize Kernel
transpose.weight.data = kernel2
# Output value
output = transpose(input)

print(output)
```

Transposed Convolution

```
tensor([[[[0., 0., 0., 1.],  
          [0., 0., 2., 3.],  
          [0., 2., 0., 3.],  
          [4., 6., 6., 9.]]]], grad_fn=<...>)  
tensor([[[[ 0.,  0.,  4.,  1.],  
          [ 0.,  0.,  2.,  3.],  
          [ 8.,  2., 12.,  3.],  
          [ 4.,  6.,  6.,  9.]]]]],  
        grad_fn=<...>)
```

CNN for Semantic Segmentation of EO Images



Dense Semantic Labeling of Subdecimeter Resolution Images With Convolutional Neural Networks, 2017

Challenges for Semantic Segmentation

- What and where
- Classes with similar spectral signature
- Inconspicuous classes

Challenges for Semantic Segmentation

- What and where
- Classes with similar spectral signature
- Inconspicuous classes

Solution for the mentioned problems

- Context

Solution for the mentioned problems

- Context
- Context at multiple scales

Solution for the mentioned problems

- Context
- Context at multiple scales
- Global context

Contraction Phase

Reduces spatial size

Increases understanding of content (“what”)

Though loses some object information due to contraction (loses “where” to some extent)

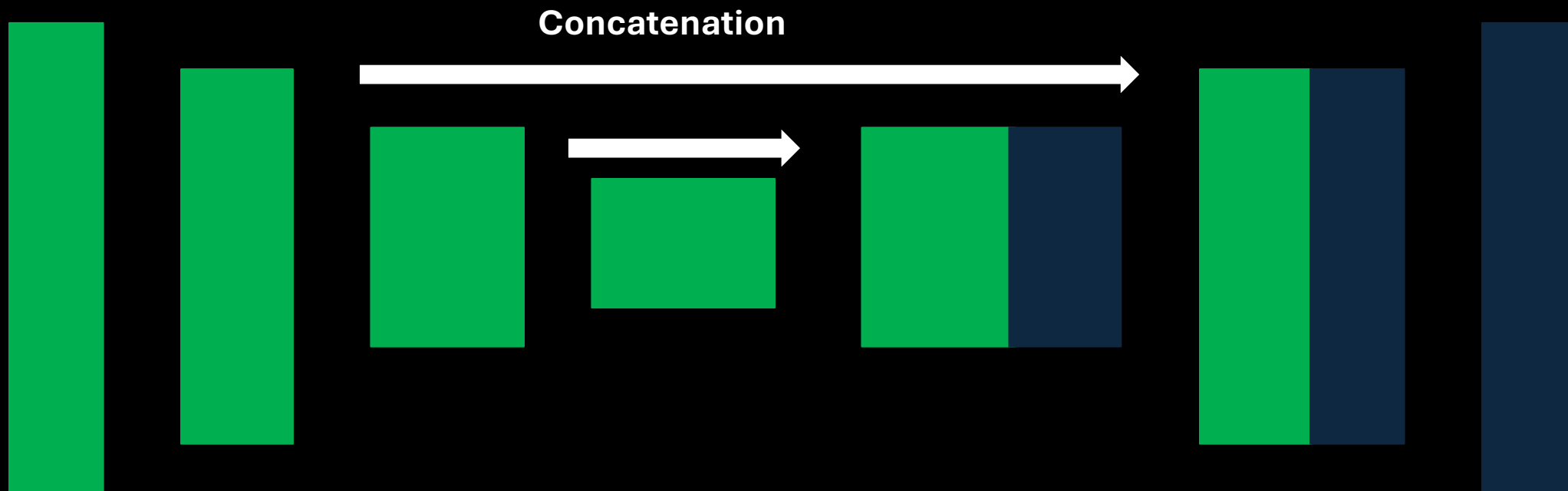
Recover “where” in expansion phase

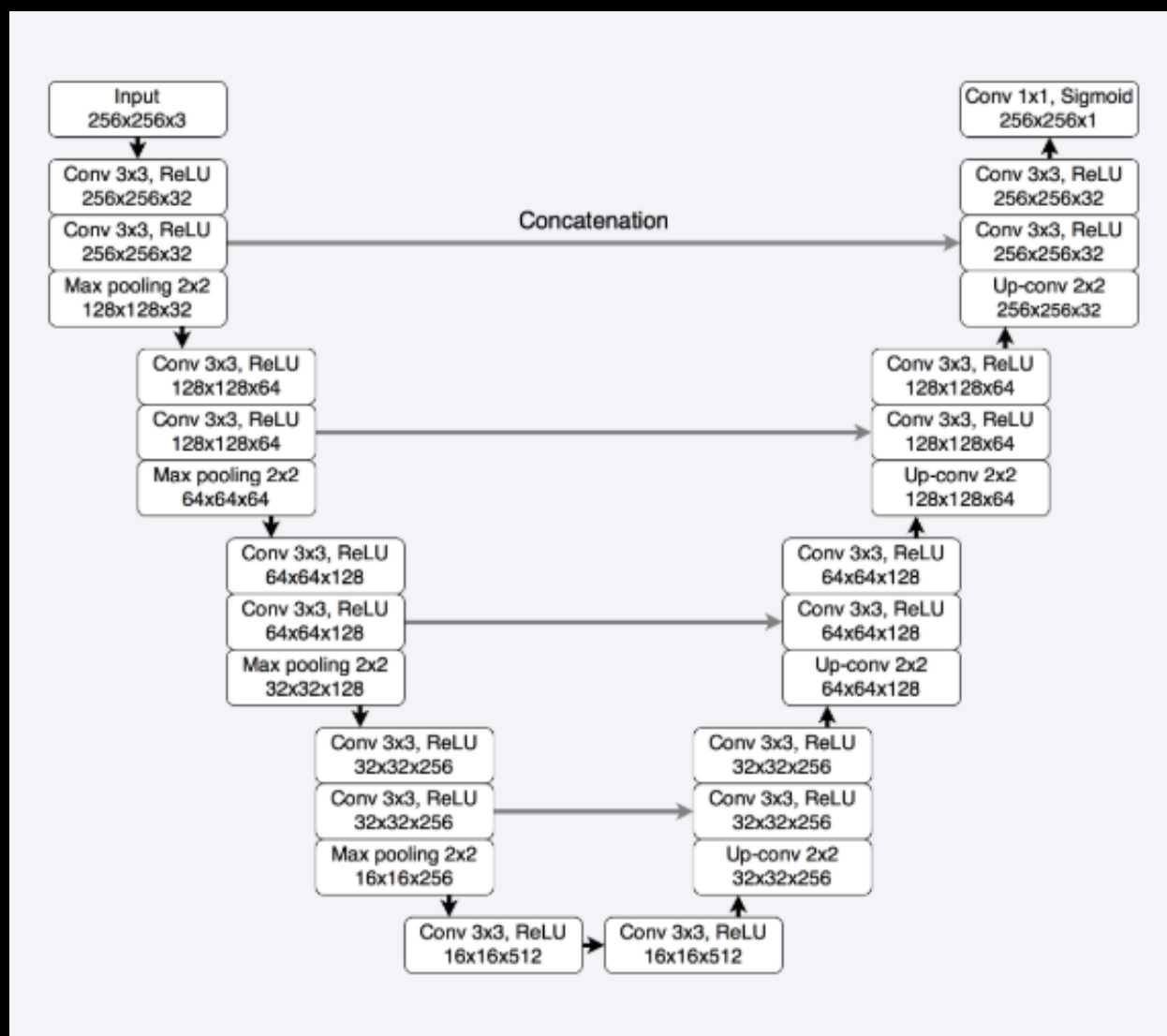
Concatenate feature maps from contraction phase during expansion phase during recovery

Helps in recovery of “where” information

Principle behind U-Net

U-Net





```
def forward(self, x):
    enc1 = self.encoder1(x)
    enc2 = self.encoder2(self.pool1(enc1))
    enc3 = self.encoder3(self.pool2(enc2))
    enc4 = self.encoder4(self.pool3(enc3))

    bottleneck = self.bottleneck(self.pool4(enc4))

    dec4 = self.upconv4(bottleneck)
    dec4 = torch.cat((dec4, enc4), dim=1)
    dec4 = self.decoder4(dec4)
    dec3 = self.upconv3(dec4)
    dec3 = torch.cat((dec3, enc3), dim=1)
    dec3 = self.decoder3(dec3)
    dec2 = self.upconv2(dec3)
    dec2 = torch.cat((dec2, enc2), dim=1)
    dec2 = self.decoder2(dec2)
    dec1 = self.upconv1(dec2)
    dec1 = torch.cat((dec1, enc1), dim=1)
    dec1 = self.decoder1(dec1)
    return torch.sigmoid(self.conv(dec1))
```

```
self.decoder4 = UNet._block((features * 8) * 2, features * 8, name="dec4")
self.upconv3 = nn.ConvTranspose2d(
    features * 8, features * 4, kernel_size=2, stride=2
)
```

Spatial Attention + UNet

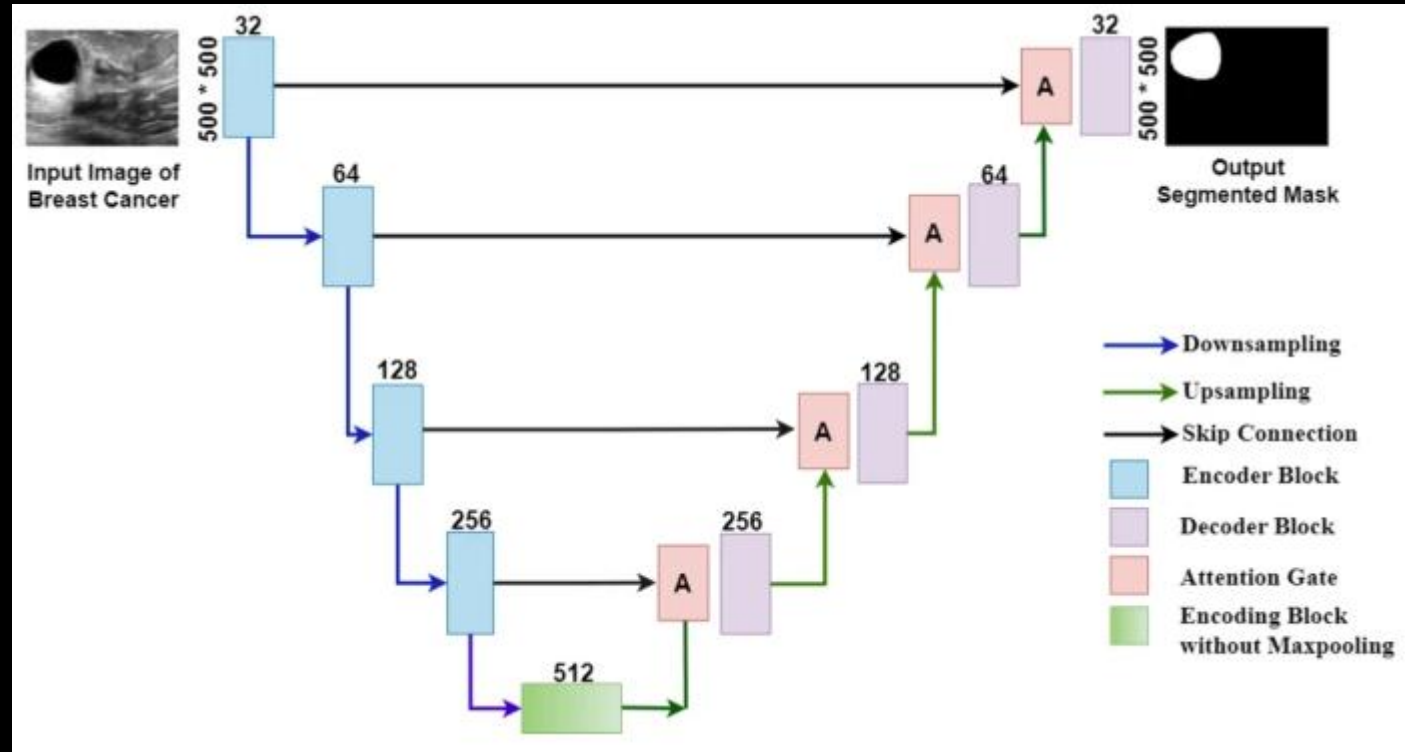


Figure from: Attention based UNet model for breast cancer segmentation using BUSI dataset, 2024

However, idea of attention based UNet is much older

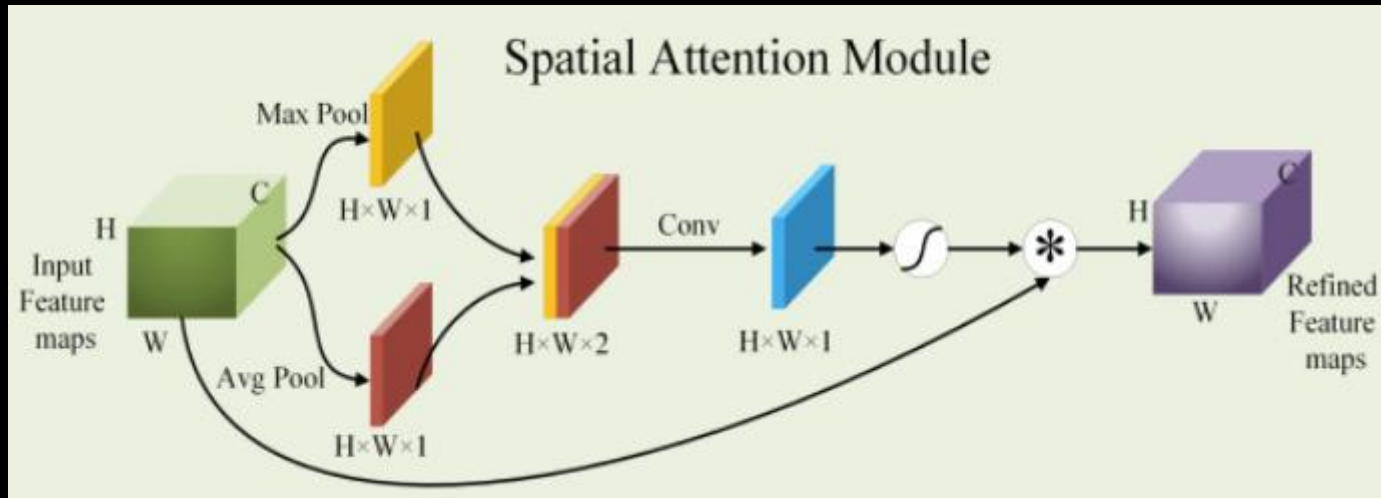
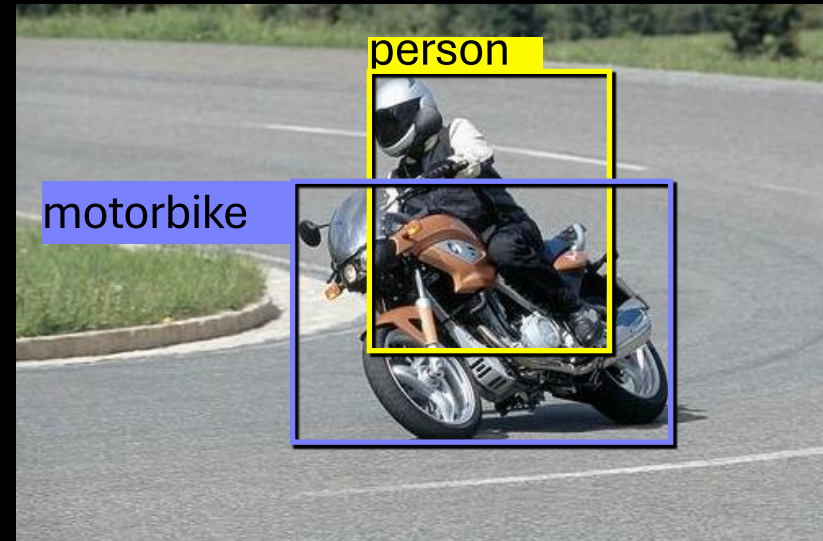


Figure from: An improved medical image segmentation framework with Channel-Height-Width-Spatial attention module

Object Detection



Two/multi-step process

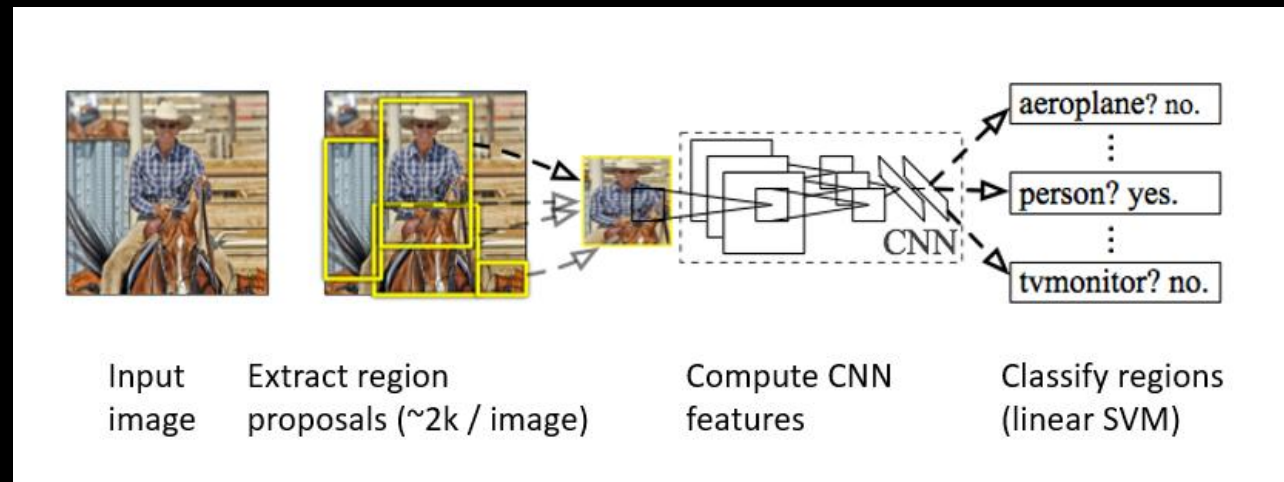
- Think about ScAI admission test

Two/multi-step process

- Decouple object proposal generation from classification
- Proposal generation – class-agnostic

RCNN

Step 1: Generation of category-independent region proposals

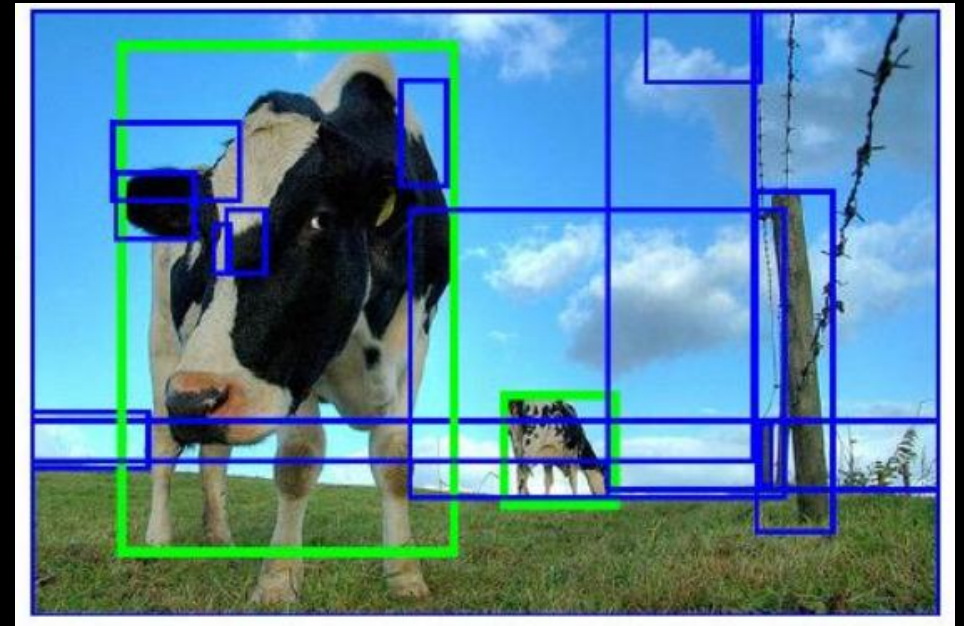


Along with
Bounding
box regression

Input image	Extract region proposals (~2k / image)	Compute CNN features	Classify regions (linear SVM)
----------------	---	-------------------------	----------------------------------

Region Proposals

- ✓ Region proposal methods take an image as input and output bounding boxes corresponding to all patches in the image that are likely to be objects.
- ✓ The region proposals can be overlapping.
- ✓ The region proposals can be noisy.



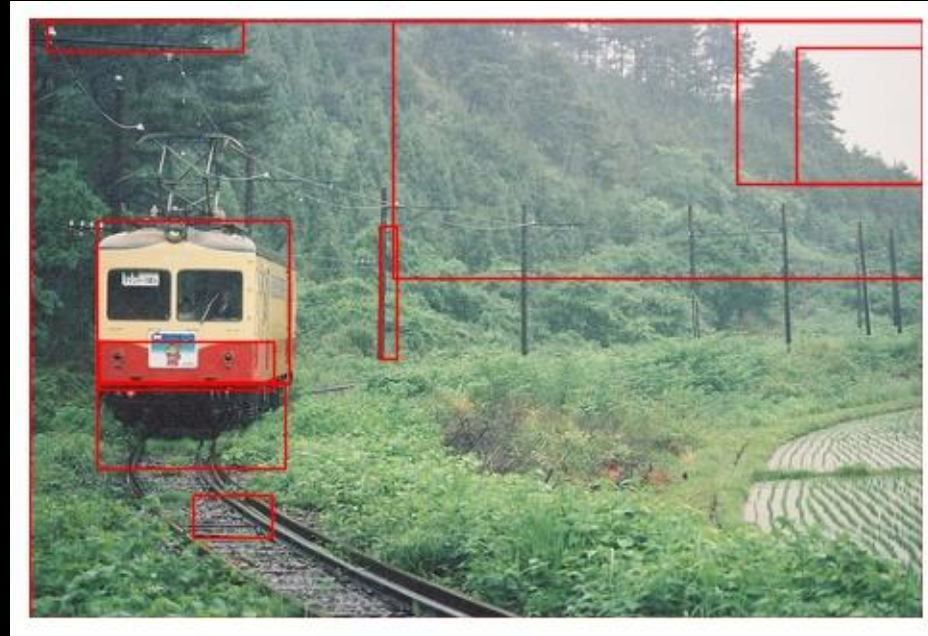
Selective Search

Similarity Measures

- ✓ Color
- ✓ Texture
- ✓ Size
- ✓ Fill

```
1 import skimage.io
2 import selective_search ←
3 import matplotlib.pyplot as plt
4 import matplotlib.patches as mpatches
5
6
7 # Load image as NumPy array from image files
8 inputPath = './inputImages/trainScene.jpg'
9 image = skimage.io.imread(inputPath)
10 image = image[:, :, 0:3]
11
12 # Run selective search using single mode
13 boxes = selective_search.selective_search(image, mode='single', random_sort=False) ←
14 boxes_filter = selective_search.box_filter(boxes, min_size=5, topN=10)
15
```

Selective Search

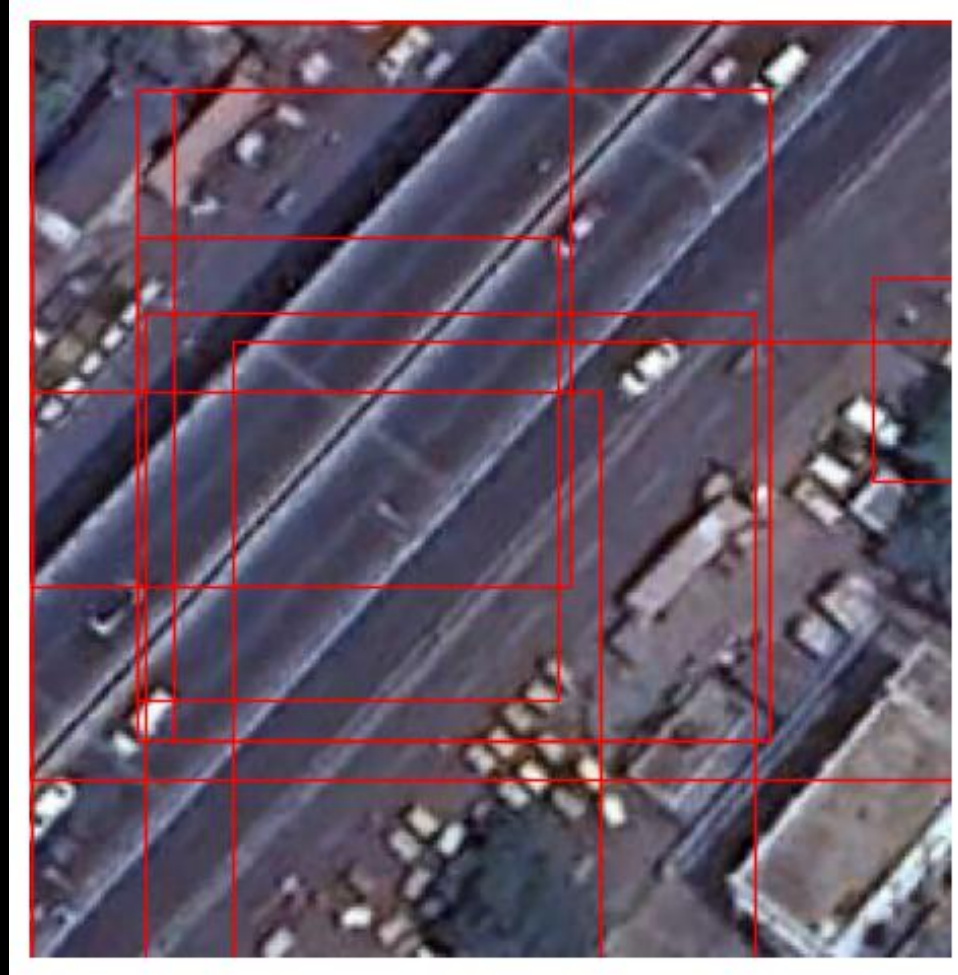


Selective Search



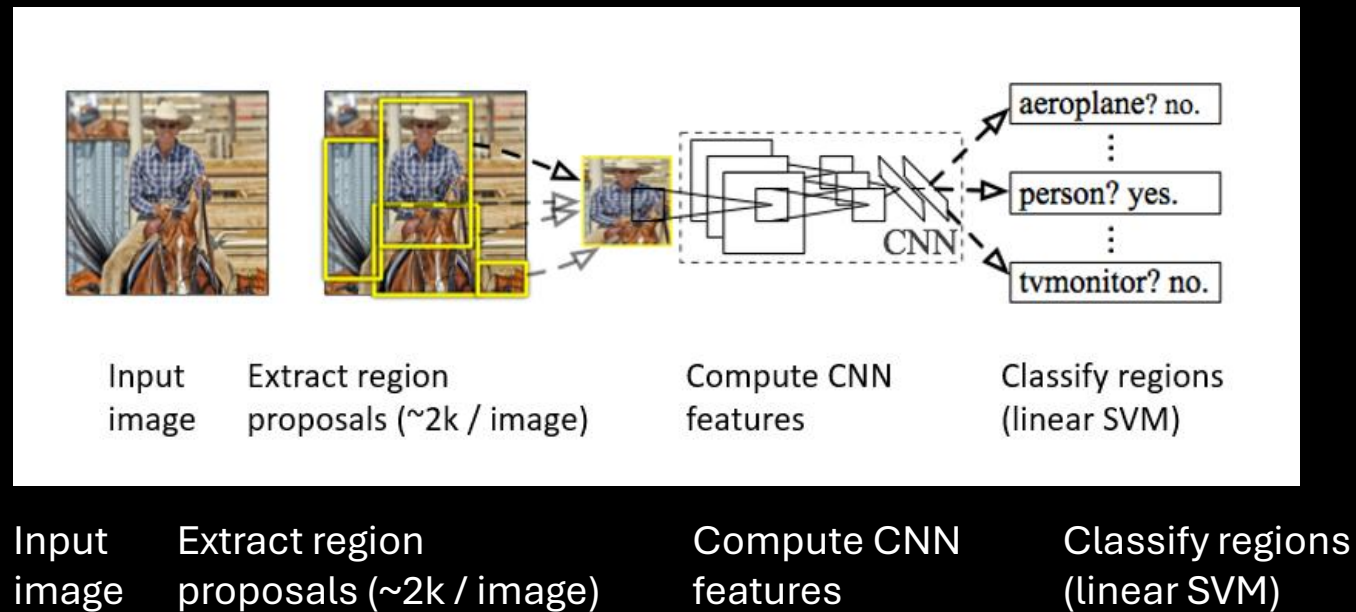
- ✓ A marine aerial image showing ships
- ✓ We just generated 10 proposals, so 1 ship was not detected

Selective Search

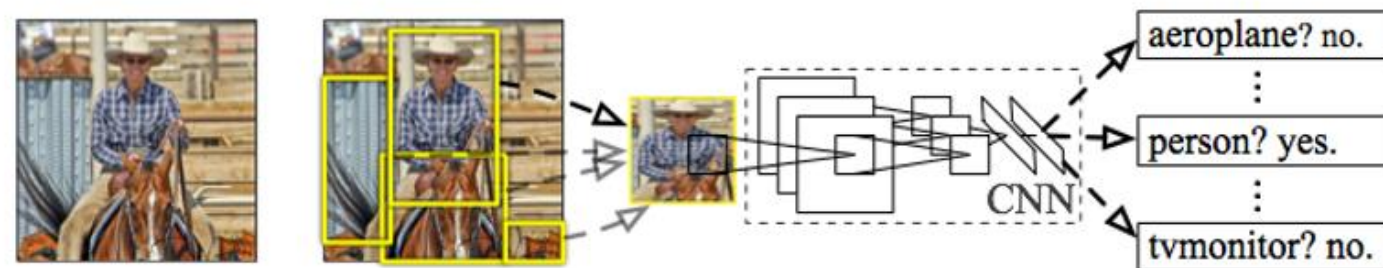


RCNN

Step 2: Extraction of fixed size feature vectors from each proposal



RCNN



Input
image

Extract region
proposals (~2k / image)

Compute CNN
features



a. Crop



227 x 227

b. Scale

RCNN

- Will a generic pre-trained CNN be good enough?

RCNN

- Will a generic pre-trained CNN be good enough?

Think about domain shift (spatial resolution differences caused by taking crops)

RCNN

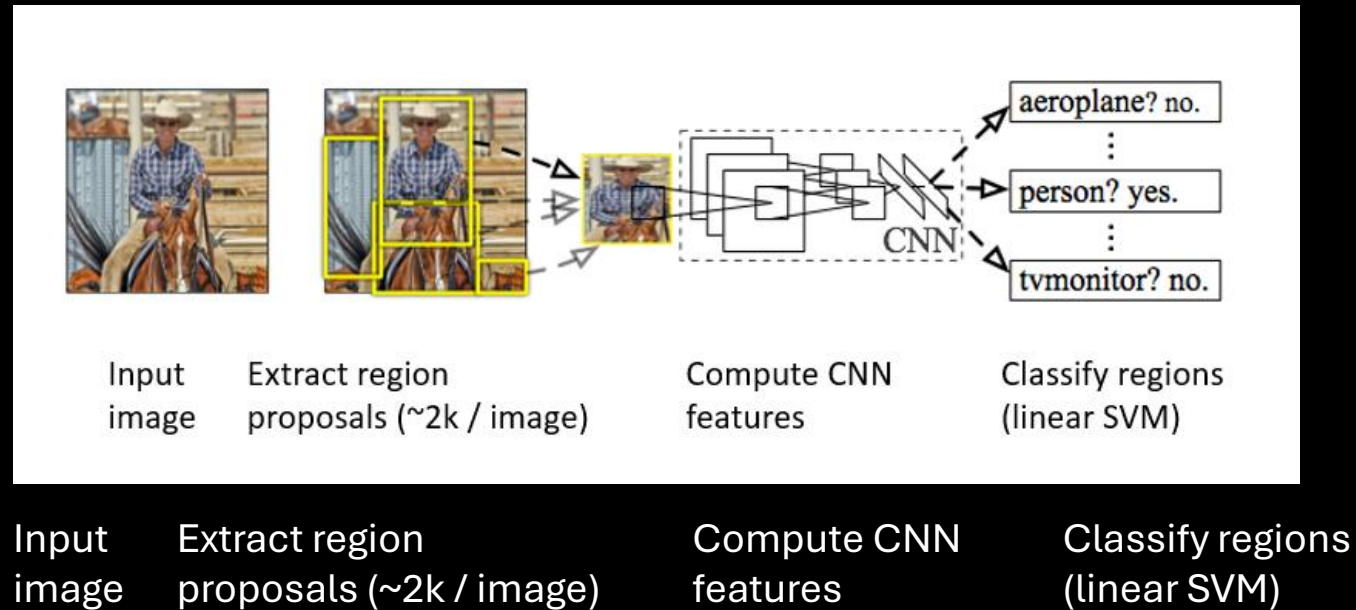
- Will a generic pre-trained CNN be good enough?

Think about domain shift (spatial resolution differences caused by taking crops)

- CNN fine-tuning on cropped images

RCNN

Step 3: Classify using a set of class-specific linear SVMs



RCNN

- SVM based classifier

RCNN – training data preparation for SVM

- An image region that tightly encloses a car is treated as a positive example, while a region corresponding purely to background is treated as a negative example.
- Ambiguity arises for regions that only partially overlap with a car.
- This ambiguity is addressed by assigning labels based on an Intersection-over-Union (IoU) overlap threshold.

RCNN – Major Drawback

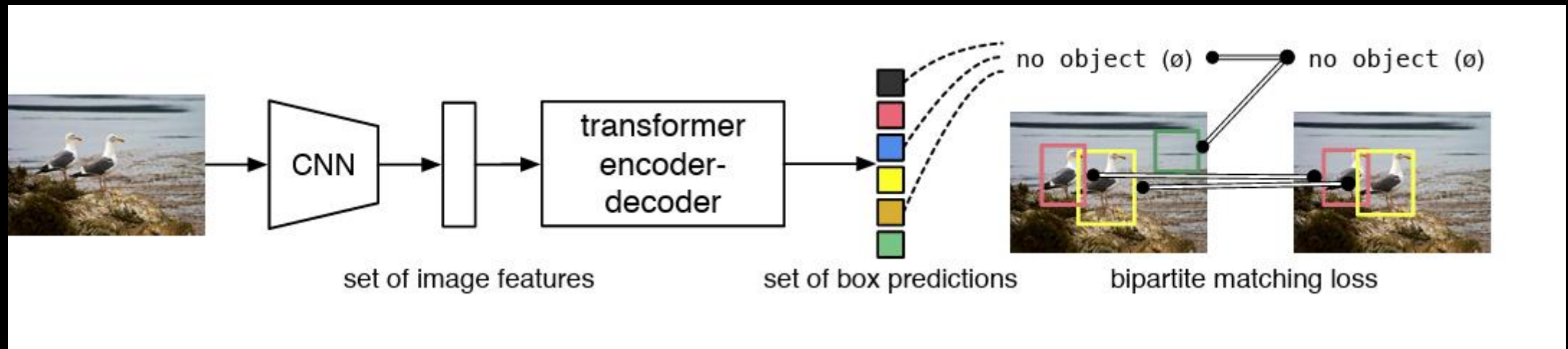
- Processes each ROI through CNN (computationally expensive).

DETR

- DEtection Transformer
- See the paper year
- Not really a ViT (as we know)

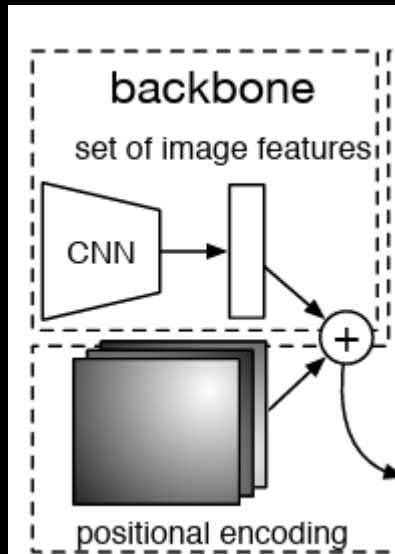
End-to-End Object Detection with Transform, 2020

DETR



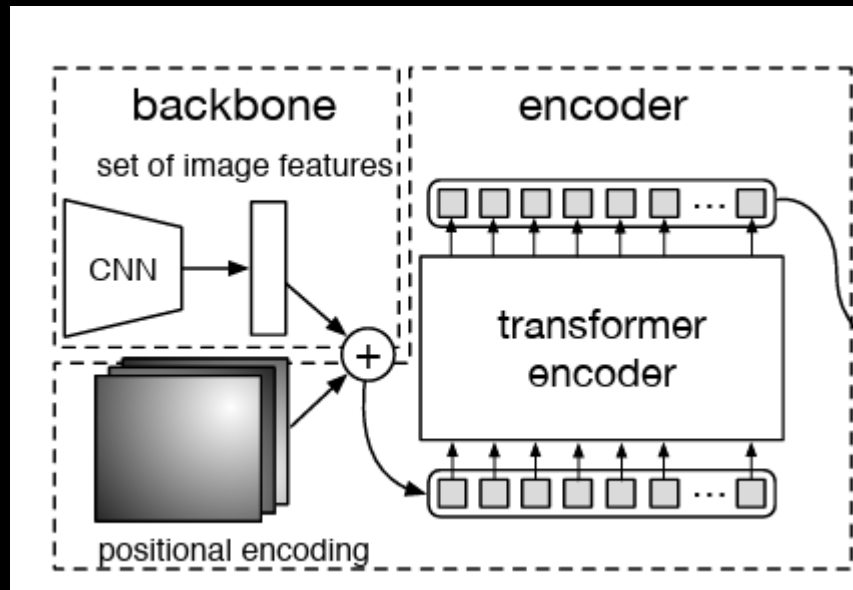
End-to-End Object Detection with Transform, 2020

DETR



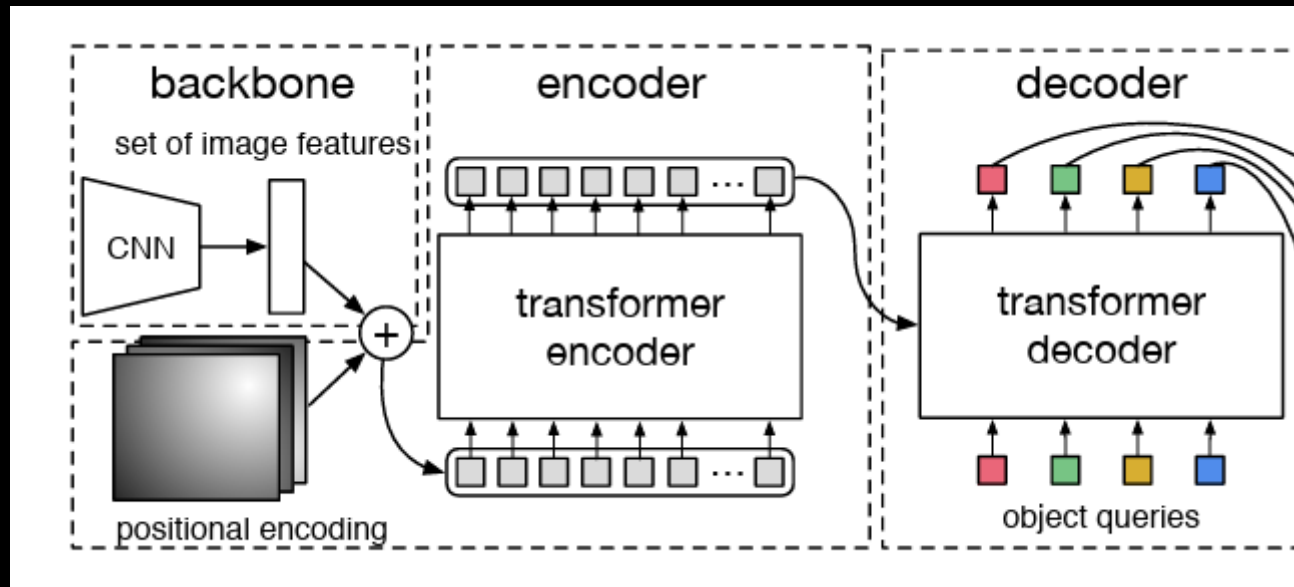
End-to-End Object Detection with Transform, 2020

DETR



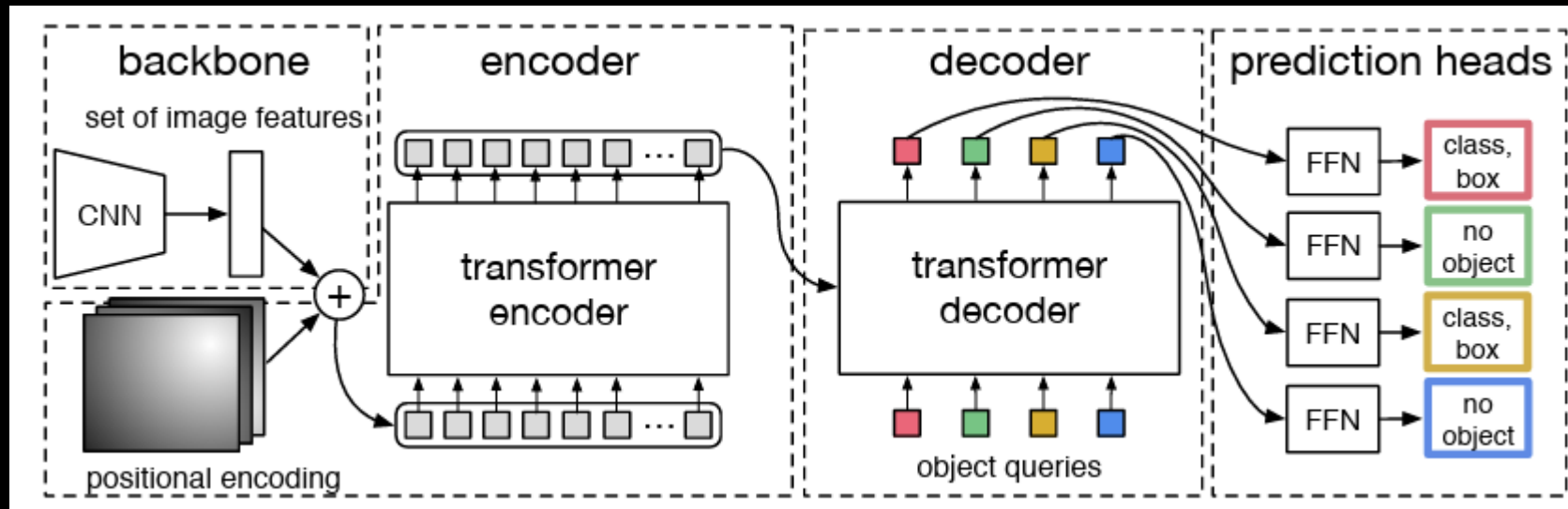
End-to-End Object Detection with Transform, 2020

DETR



End-to-End Object Detection with Transform, 2020

DETR



End-to-End Object Detection with Transform, 2020

Loss

- Matches predictions with ground-truth objects one-to-one: DETR produces a fixed number of object predictions (tokens), but an image may contain fewer objects.
- DETR assigns each real object to exactly one predicted token, ensuring that each object is detected only once and avoiding duplicate detections.

Loss

- Penalizes both wrong class and wrong location together.

Loss

- Penalizes both wrong class and wrong location together.
- Unused predictions are treated as “no object”: Since DETR outputs many predictions, those that are not matched to any real object are trained to predict “no object.” This helps the model learn when nothing is present and removes the need for post-processing like non-maximum suppression.